

GIT Report

Sean D' Almeida
Driverless Subsection
Formula Manipal
Manipal, Udipi
seanzinbox@gmail.com

ABSTRACT - Version control systems (VCS) are essential tools in modern software development; they help developers track code changes and work together. This technical report explains how Git works, focusing on its local workflow and underlying snapshot system. Key topics are: the staging area, stashing, commit structures, branching, merging and rebasing, pointer (HEAD) movement and remote synchronization using the push and pull commands. The report also covers common real-world challenges, such as handling push rejection, resolving branch divergences so on. Finally, a practical local demonstration shows how to initialize a repository, stage files, make commits, and push changes to a remote repository in Github. Keywords - Version Control Systems (VCS), Git Mechanics, Branching & Merging, Merge Conflicts, Remote Synchronization.

I. INTRODUCTION

In software development, tracking changes across a project is essential. Version Control Systems (VCS) solve this problem by keeping a history of file edits, making it easy to recover past work and allowing multiple people to code together without overwriting each other's changes. Older centralized systems stored all files on a single server, which meant developers needed an internet connection to work. Modern tools like Git are distributed version control systems. This means that every developer on a shared remote repository has a full copy of the project history saved locally on their computer.

Created by Linus Torvalds, Git manages project history differently than older systems. Instead of saving file differences line-by-line; it takes lightweight snapshot of the entire project every time changes are saved. Then, Git links these snapshots together in a timeline using secure code hashes (which are essentially references but for commits). This design makes Git fast, reliable, and efficient at handling branches for experimental code (especially when it comes to complex workflows). This report explains the fundamental features and common workflows of Git. Section II breaks down core mechanics, including staging, committing, branching, navigating history, and syncing with remotes. Section III demonstrates a step-by-step local workflow showing how to set up a repository, stage files, create commits, and push them to a remote server. Section IV concludes with a brief summary of key takeaways.

II. GIT MECHANICS

A. The Staging Area

The staging area (also called the index) acts as a middle ground between your working folder and the Git database. Instead of saving every minor edit automatically, Git requires you to choose which files you want to include in a snapshot. You can check which files are modified, staged, or untracked at any point by using the `git status` command.

B. Stashing in GIT

Often times, developers come to a crossroads: you need to switch to different branches or pull incoming commands, however your current work isn't ready for a commit; so instead of staging, git allots you to a temporary storage for your uncommitted work.

The main difference between stashing and staging comes down to intent: staging prepares changes to become a permanent part of your project's history, while stashing temporarily hides uncommitted work so you can clean your workspace without saving a permanent commit.

C. Commit Concept and Structure

In version control, a commit is a permanent, named snapshot of your project at a specific moment. Executing `git commit` takes everything currently in the staging area and records it into the repository's history database. As mentioned before, commits are extremely efficient as Git links the changes to the older file versions instead of copying them again.

Every commit contains three main pieces of information:

1. **File Snapshot:** A pointer to the project files and folders saved in that version.
2. **Author Details:** Information about who made the changes, including a timestamp and a commit message.
3. **Parent Links:** Hash addresses that point back to previous commits, forming a connected history timeline.

D. Branch Mechanics

Branching allows developers to work on new features, experiment with ideas, or fix bugs without changing the main, working code. Similar to commits, Git branches are very lightweight. A branch is not a full copy of your project folder;

it is simply a small text file that points to a specific commit hash.

When you run `git branch <branch_name>`, Git creates a new pointer at your current location in the commit history. This lets you work on separate lines of development without breaking the main branch (e.g., `main`).

E. Branch Navigation and Shortcuts

Creating a new branch pointer does not automatically move you to it. To switch your active workspace to the new branch, you use the command `git checkout <branch_name>`. This moves the HEAD pointer (which tracks what you are currently working on) to that branch. Any commits you make after checking out will only be added to that specific branch line.

Because creating a branch and immediately switching to it is such a common workflow, Git offers a shortcut command: `git checkout -b <branch_name>`

F. Merging Branch Histories

While branching allows for isolated development, isolated changes eventually need to be combined back into the primary codebase. Git provides the `git merge <branch_name>` command to integrate history from a target branch into the currently active branch (where HEAD is located).

When two branches have diverged, Git performs a three-way merge by comparing the snapshots of both branch tips and their common ancestor. This operation generates a unique **merge commit** that explicitly features two parent pointers. Through this multi-parent structure, Git preserves the non-linear history of how side experiments or bug fixes were developed in parallel before being integrated.

G. Linear Integration via Rebasing

Rebasing provides an alternative method for combining work from two separate branches. Rather than creating a multi-parent merge commit, the `git rebase <target_branch>` command takes the commits from the current branch and reapplies them individually on top of the target branch.

Underneath, Git does not move the original commits; instead, it generates brand-new commit objects with distinct hashes that contain identical code changes. The primary advantage of rebasing over merging is that it maintains a clean, linear commit history without extra merge commits.

Rebasing can be executed in two ways:

- **Non-Interactive Rebase:** Standard rebasing using `git rebase <target_branch>` automatically sequentially reapplies commits. To rebase a specific branch without manually checking it out first, Git allows specifying both branches: `git rebase <target_branch> <topic_branch>`.
- **Interactive Rebase:** Triggered with the `-i` flag (`git rebase -i <target_branch>`), this mode opens an interface allowing developers to reorder, edit, combine (squash), or drop commits prior to applying them.

H. Detached HEAD State and hash reference

During standard navigation, the HEAD pointer is attached to a branch name, which is indicated in terminal outputs by an asterisk next to the active branch (e.g., `main*`). As new commits are made, the active branch reference and HEAD move forward together automatically.

However, developers can navigate directly to any historical commit rather than a branch by checking out its unique cryptographic address (hash): `git checkout <commit_hash>`

When checking out a specific commit hash directly, Git enters a **Detached HEAD** state. In this mode, HEAD points directly to an individual commit snapshot rather than a branch pointer. While in a detached state, any new commits created will not belong to any branch, making this state useful for inspecting past code versions without accidentally modifying existing branch pointers.

I. Relative Referencing and Pointer Manipulation

Navigating Git history using full 40-character SHA-1 hashes (e.g., `fed2da64...`) can be inefficient. While Git permits short hash prefixes (typically the first 4 to 7 characters) to identify commits, relative references provide a more fluid navigation method relative to known points like HEAD or branch names.

Git provides two primary operators for relative traversal:

- **The Caret (^) Operator:** Appended to a reference (e.g., `HEAD^`), it moves the reference pointer directly up to its immediate parent commit.
- **The Tilde (~) Operator:** Followed by a number (e.g., `HEAD~3`), it traverses upward through the commit history by the specified number of parent commits.

Beyond basic workspace navigation, relative referencing allows developers to rapidly reassign branch pointers without manually checking them out. By combining the force flag (`-f`) with `git branch`, a branch pointer can be moved directly to any historical or relative commit: `git branch -f main HEAD~3`

J. Modifying History: Revert vs Reset

When code modifications need to be undone or revised, Git provides two fundamental strategies depending on whether the commit history has been shared publicly with a remote repository:

- **Local History Reset (`git reset`):** The `git reset <reference>` command rewinds branch pointers backward to a designated commit snapshot. This effectively removes newer commits from the current branch history as though they were never created. Because this operation rewrites local history, it is best suited for unpushed, local commits. Depending on how workspace modifications should be handled, Git offers three modes of reset:
 1. **Hard Reset (`git reset --hard`):** Rewinds the commit history, wipes the staging index, and completely discards all working directory

- changes. This fully destroys uncommitted code and history back to the targeted commit.
- 2. **Mixed Reset (git reset --mixed / default):** Rewinds the commit history and clears the staging area, but preserves your actual code modifications as unstaged changes in your local working directory.
- 3. **Soft Reset (git reset --soft):** Rewinds the commit pointer while keeping all code changes intact and pre-staged in the index, allowing developers to easily group and re-commit the work into a single unified snapshot.
- **Public History Modification (git revert):** To safely undo changes that have already been pushed to a remote repository, `git revert <commit_hash>` is used. Instead of deleting past commits from the timeline, git revert calculates the inverse changes of the specified commit and creates a brand-new commit that cancels them out. This preserves a complete audit trail without rewriting shared project history.

K. Reorganizing Commits

In complex workflows, developers often need to reorder, select, or transfer individual commits between branches. Git provides two primary mechanisms for commit relocation:

- **Targeted Commit Copying (git cherry-pick):** The `git cherry-pick <commit_hash_1> <commit_hash_2>` command copies specific commit snapshots from anywhere in the repository and applies them sequentially onto the current branch tip. This is particularly useful when extracting a single bug fix from an experimental branch without merging the entire branch.
- **Interactive Rebasing (git rebase -i):** Running `git rebase -i <target_reference>` launches an interactive text interface (in case of Git: by default wim) listing a range of commit hashes. Through this interface, developers can explicitly reorder commits, squash multiple small commits into a single unified snapshot, modify commit messages, or drop unnecessary commits entirely before applying them to the target timeline

L. Remote Synchronization Mechanics

Remote repositories (eg. on Github) act as centralized or peer-hosted hubs that enable collaboration across distributed environments. Working with remote repositories requires transferring commit objects and pointer data between local and remote states using four primary commands:

1. **Repository Cloning (git clone):** The “`git clone <repository_url>`” command initializes a local directory, copies the complete object database from the remote host, and instantiates tracking pointers (such as `origin/main`) that reflect the remote repository's state at the time of download.
2. **Fetching Updates (git fetch):** The `git fetch` command connects to the remote repository and downloads any new commit objects or branches missing from the local database. It updates remote-tracking references (e.g., `origin/main`), but does **not** modify active local working branches or merge code automatically.
3. **Fetching and Merging (git pull):** Because downloading remote changes and integrating them locally is a standard workflow, Git provides `git pull`. Executing `git pull` is an atomic operation equivalent to running `git fetch` followed by `git merge origin/<branch_name>`.
4. **Publishing Changes (git push):** The `git push` command uploads local commit snapshots to the remote repository and updates the remote branch reference to point to the new tip commit, making those changes accessible to other collaborators.

M. Remote Divergence and Push Rejections

A common issue in collaborative Git workflows is **remote divergence**. Divergence occurs when a developer attempts to push local commits to a remote branch, but another collaborator has already pushed different commits to that same remote branch in the meantime.

Remote: `C0 <-- C1 <-- C2` (Pushed by collaborator)

Local: `C0 <-- C1 <-- C3` (Your local work)

In this state, the local branch and remote branch have diverged from a common ancestor commit (C1). When executing `git push`, Git detects that the remote reference cannot be updated via a simple fast-forward operation and rejects the push request ("**Remote Rejected!**").

To resolve remote rejection, Git forces the developer to integrate the latest remote changes into their local workspace before publishing:

- **Fetch and Merge Workflow:** The developer fetches remote updates and merges them (`git fetch` followed by `git merge origin/main`), creating a 2-parent merge commit before pushing.

- **Shorthand Rebase Workflow** (`git pull --rebase`): To avoid cluttering repository history with extra merge commits, developers frequently execute `git pull --rebase`. This shorthand command fetches the remote commits (`git fetch`) and immediately reapplies the local commits on top of the fetched remote tip (`git rebase origin/main`), resulting in a clean, linear history ready for immediate pushing.

N. Protected Branch Constraints and Recovery Mechanics

In software development, shared primary branches (such as `main` or `release`) are protected by administrative rules on remote hosts (e.g., GitHub, Azure DevOps, or GitLab). These protections restrict direct pushes to ensure code quality through mandatory peer reviews and automated continuous integration (CI) checks.

If a developer accidentally commits changes directly to their local `main` branch and attempts to execute `git push`, the remote server rejects the operation with a branch protection error: “! [remote rejected] `main -> main` (TF402455: Pushes to this branch are not permitted; you must use a pull request to update this branch.)”

This error creates a state where local `main` has progressed ahead of remote `main` with unapproved commits, blocking normal pushing and preventing clean synchronization.

O. Recovery workflow for Protected Branch Misallocations

To resolve this issue without losing local changes or violating branch protections, developers must migrate the unpushed commits off local `main` and onto a newly feature branch, then reset local `main` back to match the remote state.

This recovery is executed through a three-step sequence:

1. Instantiate a Feature Branch:

Create a new branch at the current `HEAD` commit to preserve all local work:

```
git branch feature-work
```

This creates `feature-work` pointing to your recent local commits while keeping `HEAD` on `main`.

2. Rewind Local Main (`git reset`):

Force local `main` backward to match the upstream remote tracking branch using relative references or remote pointers:

```
git reset --hard origin/main
```

This resets local `main` back to the exact commit authorized on the remote server without deleting the commits now anchored by `feature-work`.

3. Switch, Push, and Open a Pull Request:

Check out the newly instantiated feature branch and push it to the remote repository:

```
git checkout feature-work
```

```
git push -u origin feature-work
```

Once pushed, the developer can submit a formal **Pull Request (PR)** or **Merge Request (MR)** via the repository web interface, adhering to code review policies without corrupting local or remote branch histories.

P. Milestone Markers via Git Tags

While branches serve as dynamic, moving pointers that advance as new commits are recorded, software projects also require static references to mark release milestones or stable builds (e.g., `v1.0`, `v2.1-beta`). Git provides **tags** specifically to fulfill this requirement.

Unlike branches, tags are fixed anchors within the commit graph that never move automatically when new commits are created. A tag cannot be checked out to receive ongoing commits directly; instead, it remains permanently attached to a designated commit snapshot.

To create an explicit milestone anchor, the `git tag` command is executed by specifying the tag name followed by the target commit hash: `git tag v1.0 c1a2b3`

If no commit hash is specified, Git attaches the tag to the current `HEAD` location.

Q. Relative Distance Inspection via `git describe`

Because tags serve as recognized milestone anchors across a codebase, Git provides a specialized inspection command called `git describe` to determine how far the active workspace or target reference has diverged from the nearest preceding tag.

Executing `git describe <ref>` evaluates the distance between the specified reference (or `HEAD` if omitted) and the closest tag anchor in the commit history line:

```
git describe HEAD
```

The output returned by Git follows a standardized structural format:

```
<tag>-<numCommits>-g<hash>
```

- **<tag>**: The name of the most recent tag found in the commit lineage prior to the target reference.
- **<numCommits>**: The precise number of commits that the target reference sits ahead of that tag anchor.
- **g<hash>**: The letter 'g' (signifying Git) followed by the short cryptographic hash prefix of the target commit.

This output allows developers to quickly identify their relative position within a large release timeline, making git describe an essential tool for build systems, automated versioning scripts, and bug triage.

III. PRACTICAL DEMONSTRATION

A. Setup and Repository Initialization

To demonstrate core version control concepts in a real local environment, a Python application file (TPOne3.py) located inside the project directory (C:\Users\Sean\.vscode\Python\Task1) was initialized with Git tracking. Using the Git Bash console, the base application state was staged and committed directly to the main branch to establish an initial project baseline.

```
Sean@DESKTOP-03VP2E9 MINGW64 ~/.vscode/Python/Task1 (main)
$ git checkout -b feature-branch
Switched to a new branch 'feature-branch'

Sean@DESKTOP-03VP2E9 MINGW64 ~/.vscode/Python/Task1 (feature-branch)
$ echo "print('Feature branch edit')" >> TPOne3.py

Sean@DESKTOP-03VP2E9 MINGW64 ~/.vscode/Python/Task1 (feature-branch)
$ git commit -am "feat: update binary search on feature branch"
warning: in the working copy of 'TPOne3.py', LF will be replaced by CRLF the next time Git touches
[feature-branch f47b975] feat: update binary search on feature branch
1 file changed, 1 insertion(+), 1 deletion(-)
```

B. Feature Branching and Code Modification

To simulate a isolated feature development workflow without breaking the stable release, a secondary branch named feature-branch was created and checked out using the git checkout -b shortcut.

An edit was added to TPOne3.py on this isolated branch using command-line string appending via the echo command. The changes were then committed locally using git commit -am, which automatically staged the tracked modified file while recording a commit message.

```
Sean@DESKTOP-03VP2E9 MINGW64 ~/.vscode/Python/Task1 (feature-branch)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

Sean@DESKTOP-03VP2E9 MINGW64 ~/.vscode/Python/Task1 (main)
$ echo "print('Main branch edit')" >> TPOne3.py

Sean@DESKTOP-03VP2E9 MINGW64 ~/.vscode/Python/Task1 (main)
$ git commit -am "feat: update binary search on main branch"
warning: in the working copy of 'TPOne3.py', LF will be replaced by CRLF the next time Git touches it
[main 6ac86ba] feat: update binary search on main branch
1 file changed, 1 insertion(+), 1 deletion(-)
```

C. Triggering an Intentional Merge Conflict

To set up a manual conflict scenario, the local workspace was switched back to main. On main, a conflicting edit was made to the exact same file location in TPOne3.py and committed

When attempting to integrate the feature branch changes into main via git merge feature-branch, Git stopped the automatic merge process and flagged a content conflict due to divergent histories on the same lines of code.

```
Sean@DESKTOP-03VP2E9 MINGW64 ~/.vscode/Python/Task1 (main)
$ git merge feature-branch
Auto-merging TPOne3.py
CONFLICT (content): Merge conflict in TPOne3.py
Automatic merge failed; fix conflicts and then commit the result.
```

D. Manual Conflict Resolution and Merge Finalization

The conflicting file was opened in Visual Studio Code. Git highlighted the overlapping changes using standard conflict markers (<<<<<<< HEAD, =====, and >>>>>>) feature-branch).

To resolve the issue, the conflict markers were manually stripped out, and the Python logic was cleaned up so both execution lines were formatted on separate lines without causing syntax errors.

After saving TPOne3.py, the resolved file was staged using git add and finalized with a formal merge commit. The complete repository history was then pushed to the GitHub remote server.

```
Sean@DESKTOP-03VP2E9 MINGW64 ~/.vscode/Python/Task1 (main|MERGING)
$ git add TPOne3.py

Sean@DESKTOP-03VP2E9 MINGW64 ~/.vscode/Python/Task1 (main|MERGING)
$ git commit -m "fix: resolve merge conflict between main and feature-branch"
[main 8caada6] fix: resolve merge conflict between main and feature-branch

Sean@DESKTOP-03VP2E9 MINGW64 ~/.vscode/Python/Task1 (main)
$ git push origin main
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 16 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (9/9), 855 bytes | 427.00 KiB/s, done.
Total 9 (delta 7), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (7/7), completed with 2 local objects.
To https://github.com/LonTec/Sean_Driverless_Taskphase
  edd6e9b..8caada6 main -> main
```

E. Historical inspection and tracking state

To inspect the internal operational log of the repository, git relog and git log --oneline --graph --all were executed. Unlike standard git log, git relog tracks head movement across commits, branch checkouts, and merge resets, serving as definitive evidence of local operations.

```
Sean@DESKTOP-03VP2E9 MINGW64 ~/.vscode/Python/Task1 (main)
$ git log --oneline --graph --all
* f47b975 (feature-branch) feat: update binary search on feature branch
| 6ac86ba feat: update binary search on main branch
/
edd6e9b Add Task 1 python programs
878d24c Initial commit

Sean@DESKTOP-03VP2E9 MINGW64 ~/.vscode/Python/Task1 (main)
$ git relog
f47b975 (HEAD -> main, origin/main) HEAD@{0}: commit: style: clean up formatting and remove duplicate function
calls
8caada6 HEAD@{1}: commit (merge): fix: resolve merge conflict between main and feature-branch
6ac86ba HEAD@{2}: commit: feat: update binary search on main branch
edd6e9b HEAD@{3}: checkout: moving from feature-branch to main
f47b975 (feature-branch) HEAD@{4}: commit: feat: update binary search on feature branch
edd6e9b HEAD@{5}: checkout: moving from main to feature-branch
edd6e9b HEAD@{6}: pull origin main --rebase (finish): returning to refs/heads/main
edd6e9b HEAD@{7}: pull origin main --rebase (pick): Add Task 1 python programs
878d24c HEAD@{8}: pull origin main --rebase (start): checkout 878d24c0f8e8ef660585465259976cb6648674c6
edd6e9b HEAD@{9}: commit (initial): Add Task 1 python programs
```

IV. CONCLUSION

This report covered topics from the core basis of VCS to the practical execution on Git (Bash) with remote repositories being established on Github. Through this report we also covered the essentials of Git Mechanics such as staging, committing, rebasing, force pushes and so on that takes into consideration complex workflow.

Key takeaways from this report are:

- **Coverage of workflow deeper than surface level:** Covered commands such as git commit, git checkout while also covering force statements such as git branch -f main, rebase statements and remote repository commands.
- **Merging vs Rebasing:** While merging preserves exact parent commit history, it introduces visual clutter especially in complex workflows. Rebasing and cherry-picking, while do not conserve the exact parent history provide an alternative through linear commit history.
- **Simulated and Real-World application:** While interactive tools such as learngitbranching.js.org help with understanding the underlying mechanics of git, applying these concepts in a live terminal (via git bash or powershell) introduces real world nuances such as line formats(<<<<<<< HEAD,)

While comprehensive, this report selectively focused on core daily workflows. Ultimately, version control is not just a safety net for backing up code, but a vital collaboration framework essential for modern software engineering.

ACKNOWLEDGMENT

I would like to express my sincere gratitude to my seniors at Formula Manipal for their guidance and encouragement throughout this task. Their insistence on technical rigor motivated me to delve deep into Git's underlying mechanics and complete this report.

REFERENCES

- [1] A. MAGALHAES, "LEARN GIT BRANCHING," *LEARNGITBRANCHING.JS.ORG*, 2023. [ONLINE]. AVAILABLE: [HTTPS://LEARNGITBRANCHING.JS.ORG](https://learngitbranching.js.org)
- [2] C. SCHAFER, "GIT & GITHUB TUTORIAL FOR BEGINNERS," *YOUTUBE*, 2017. [ONLINE]. AVAILABLE: [HTTPS://WWW.YOUTUBE.COM/USER/SCHAFER5](https://www.youtube.com/user/schafers5)
- [3] B. RYAN, "GIT GUD: A PRAGMATIC GUIDE TO GIT," *GITGUD.IO*, 2020. [ONLINE]. AVAILABLE: [HTTPS://GITGUD.IO](https://gitgud.io)